

ESCUELA DE INGENIERÍA INFORMÁTICA DE OVIEDO

Arquitectura de Computadores

Curso 2025-2026

Teamwork

Díaz Mendaña, Diego - UO301887

García Pernas, Pablo - UO300167

Rama García, Mateo - UO300710

Suárez Fernández, Fernando - UO300028

Grupo de prácticas: PL.1 - B

Titulación: PCEO Informática y Matemáticas

26 de noviembre de 2025

Índice

1. Introducción	2
2. Máquina virtual	2
3. Desarrollo del algoritmo	2
3.1. Comprobaciones e inicialización	3
3.2. Implementación del algoritmo	3
4. Desarrollo del Proyecto	5
4.1. Fase 1: Implementación Monohilo	5
4.2. Fase 2: Optimización SIMD y Multihilo	5
4.2.1. Optimización SIMD	5
4.2.2. Paralelización Multihilo	7
5. Resultados y Análisis	9
5.1. Medición del rendimiento	9
5.2. Análisis de la versión multihilo	10
5.3. Análisis de la versión SIMD	10
A. Flujo de trabajo y metodología de desarrollo	11
A.1. Metodología	11

1. Introducción

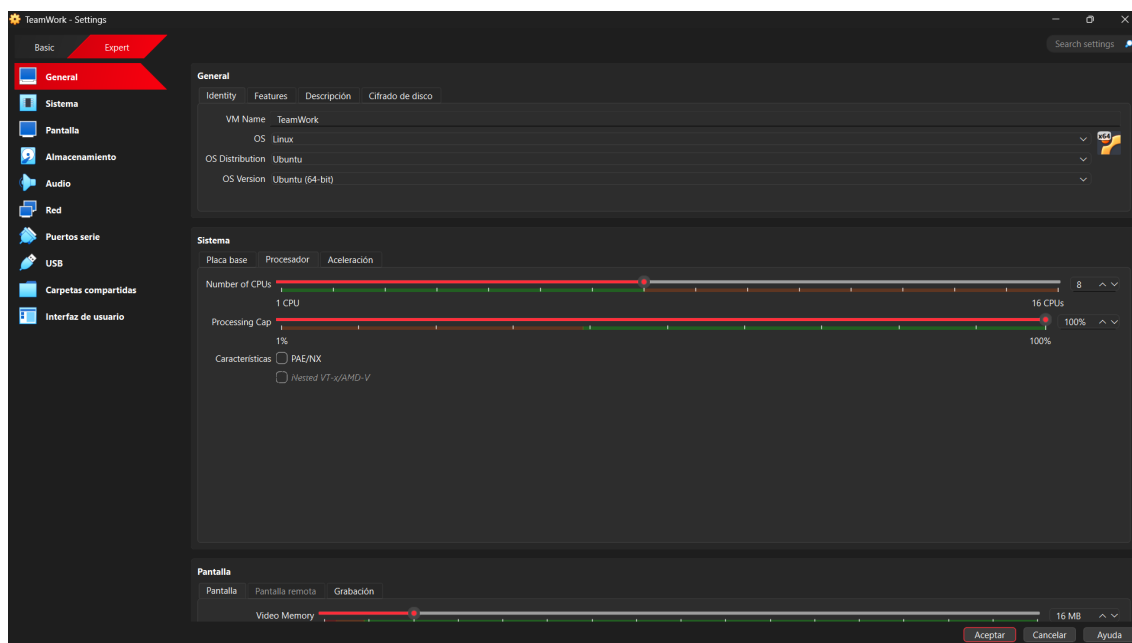
La presente memoria recoge el desarrollo del proyecto correspondiente a la asignatura *Arquitectura de Computadores*, impartida durante el curso académico 2025-2026 en la Escuela de Ingeniería Informática de Oviedo. El objetivo del proyecto es aplicar los conocimientos adquiridos en la asignatura para diseñar e implementar un filtro de imágenes en lenguaje C/C++. Asimismo, se realiza un análisis del rendimiento del filtro y una comparación entre las versiones monohilo y multihilo.

El trabajo se ha estructurado en distintas fases progresivas:

- **Entrega 1 (Monohilo):** Implementación básica del filtro de imágenes en un único hilo de ejecución.
- **Entrega 2 (SIMD y Multihilo):** Optimización del filtro mediante el uso de instrucciones SIMD y la paralelización con múltiples hilos.

2. Máquina virtual

Para el desarrollo del proyecto se utilizó la máquina virtual proporcionada por los profesores, a continuación se muestra una captura de pantalla con la configuración del sistema:



3. Desarrollo del algoritmo

Se desarrolló el algoritmo de inversión en blanco y negro (*Algoritmo número #3*). El desarrollo se realizó utilizando la máquina virtual proporcionada por el profesorado, junto con la plantilla base facilitada.

3.1. Comprobaciones e inicialización

Para la carga de la imagen, previo a su procesamiento, se empleó la carga de `CImg` controlando las posibles excepciones de lectura que puede producir dicho método:

```
1 CImg<data_t> srcImage;
2 try {
3     srcImage = CImg<data_t>(SOURCE_IMG);
4 } catch (CImgException& e) {
5     fprintf(stderr, "ERROR: la imagen '%s' no existe.\n", SOURCE_IMG);
6     exit(EXIT_FAILURE);
7 }
```

En caso de que `CImg` lance una excepción, se *cacheará*, avisará al usuario y finalizará el programa de forma controlada. En caso contrario, entonces se crea un objeto `CImg` con el que se trabajará posteriormente.

Asimismo, se modificó el tipo de dato empleado de `float` a `double`. Se definieron también los pesos para las componentes de color y el brillo máximo, como se muestra a continuación:

```
1 #define R_WEIGHT 0.3
2 #define G_WEIGHT 0.59
3 #define B_WEIGHT 0.11
4 #define MAX_BRIGHTNESS 255.0
```

Para medir el tiempo de ejecución, se emplearon las funciones `clock_gettime()` con los parámetros `CLOCK_REALTIME` y las direcciones de memoria de las variables. En caso de error, se muestra un mensaje y se finaliza la ejecución. Se calcula el tiempo transcurrido y se muestra por la salida estándar:

```
1 elapsedTime = (tEnd.tv_sec - tStart.tv_sec) +
2               (tEnd.tv_nsec - tStart.tv_nsec) / 1e+9;
3 printf("Elapsed time (%d repetitions): %.6f seconds\n",
4        N_ITERATIONS, elapsedTime);
```

3.2. Implementación del algoritmo

El algoritmo implementado corresponde al filtro de inversión en escala de grises. Para cada píxel de la imagen, la función `filter` calcula una luminosidad ponderada a partir de las componentes RGB. Para ello, se emplean los coeficientes definidos previamente. El valor resultante se invierte respecto al brillo máximo y se asigna a las tres componentes del píxel en la imagen de salida.

```
1 void filter (filter_args_t args) {
2     for (uint i = 0; i < args.pixelCount; i++) {
3         data_t L = *(args.pRsrc + i) * R_WEIGHT +
4                   *(args.pGsrc + i) * G_WEIGHT +
5                   *(args.pBsrc + i) * B_WEIGHT;
```

```

6
7     L = MAX_BRIGHTNESS - L;
8
9     *(args.pRdst + i) = L;
10    *(args.pGdst + i) = L;
11    *(args.pBdst + i) = L;
12 }
13 }

```

Dado que el tiempo de ejecución debía situarse entre 8 y 15 segundos, se realizó un bucle de 250 iteraciones de la función `filter` para mantener el tiempo dentro del rango esperado:

```

1  for (int i = 0; i < N_ITERATIONS; i++) {
2      filter(filter_args);
3  }

```

4. Desarrollo del Proyecto

4.1. Fase 1: Implementación Monohilo

En la primera fase del proyecto se desarrolló la versión monohilo del algoritmo, centrada en la comprensión del flujo de procesamiento y en la verificación de la corrección del filtro aplicado sobre las imágenes. El proceso es el descrito en la [sección anterior](#). A continuación, se muestran la imagen original y la imagen resultante tras aplicar el filtro:



Figura 1: Imagen original



Figura 2: Imagen resultante (monohilo)

4.2. Fase 2: Optimización SIMD y Multihilo

En la segunda fase del proyecto, se optimizó el algoritmo mediante el uso de instrucciones SIMD y la paralelización con múltiples hilos. No obstante, los resultados no han mostrado una mejora significativa en el rendimiento, que posteriormente se analizará en detalle.

4.2.1. Optimización SIMD

Para la optimización SIMD, se ha empleado la librería `immintrin.h` que proporciona acceso a las instrucciones SSE y AVX. Se han empleado paquetes de 128 bits (`__m128d`) para procesar dos valores `double` simultáneamente. El objetivo es aprovechar el paralelismo de datos a nivel de instrucción para reducir el número de iteraciones necesarias procesando múltiples píxeles en cada ciclo.

Para la inicialización de los vectores, las constantes de algoritmo se cargaron replicando el valor en ambos elementos del vector:

```
1 typedef __m128d simd_t;  
2  
3 const simd_t r_weight = _mm_set1_pd(R_WEIGHT);  
4 const simd_t g_weight = _mm_set1_pd(G_WEIGHT);  
5 const simd_t b_weight = _mm_set1_pd(B_WEIGHT);  
6 const simd_t max_brightness = _mm_set1_pd(MAX_BRIGHTNESS);
```

Para tener un acceso eficiente a los datos, se utiliza `_mm_malloc()` con alineamiento a 16 bytes (tamaño de `_m128d`). También se asegura el tamaño del buffer de destino para que tenga capacidad suficiente:

```
1 uint pixelCountAligned = ((filter_args.pixelCount + ITEMS_PER_PACKET - 1) /
    ITEMS_PER_PACKET) * ITEMS_PER_PACKET;
2 pDstImage = (data_t *) _mm_malloc (pixelCountAligned * nComp *
    sizeof(data_t), 16);
```

El bucle principal del filtro procesa la imagen en paquetes de dos píxeles. Para cada paquete, se cargan los valores RGB mediante `_mm_loadu_pd()`, se calcula la luminosidad ponderada con `_mm_mul_pd()` y `_mm_fmadd_pd()` y finalmente se invierte con `_mm_sub_pd()`. Los resultados se almacenan directamente en las tres componentes de salida:

```
1 const uint nPackets = (args.pixelCount / ITEMS_PER_PACKET);
2 simd_t vr, vg, vb, L;
3
4 for(uint i = 0; i < nPackets; i++){
5     const uint pos = i * ITEMS_PER_PACKET;
6
7     vr = _mm_loadu_pd(args.pRsrc + pos);
8     vg = _mm_loadu_pd(args.pGsrc + pos);
9     vb = _mm_loadu_pd(args.pBsrc + pos);
10
11     L = _mm_mul_pd(vr, r_weight);
12     L = _mm_fmadd_pd(vg, g_weight, L);
13     L = _mm_fmadd_pd(vb, b_weight, L);
14     L = _mm_sub_pd(max_brightness, L);
15
16     *(simd_t*)(args.pRdst + pos) = L;
17     *(simd_t*)(args.pGdst + pos) = L;
18     *(simd_t*)(args.pBdst + pos) = L;
19 }
```

Dado que el número total de píxeles puede no ser múltiplo exacto de 2, hay un bucle residual que procesa los píxeles restantes de forma escalar, empleando la misma lógica que en la versión monohilo. Finalmente, se libera la memoria alineada con `_mm_free()` para evitar fugas de memoria. A continuación, se muestran las imágenes resultantes:



Figura 3: Imagen original



Figura 4: Añadir imagen SIMD

4.2.2. Paralelización Multihilo

Para la paralelización multihilo, se utilizó la librería `pthread.h` para crear y gestionar múltiples hilos de ejecución. El objetivo es dividir la carga de trabajo entre varios núcleos del procesador, empleando 8 hilos de ejecución (`NUM_THREADS = 8`) para distribuir la imagen en fragmentos que se pueden procesar de forma concurrente.

La estructura `filter_args_t` se amplió con dos campos adicionales (`start` y `end`) que delimitan el rango de píxeles que cada hilo debe procesar, para trabajar sobre porciones independientes de la imagen:

```
1  typedef struct {
2      data_t *pRsrc;
3      data_t *pGsrc;
4      data_t *pBsrc;
5      data_t *pRdst;
6      data_t *pGdst;
7      data_t *pBdst;
8      uint pixelCount;
9      uint start, end;
10 } filter_args_t;
```

La función `thread_filter()` se ejecuta en cada hilo, donde recibe los punteros a los datos y rango de píxeles a procesar. Primero ajusta los punteros aplicando el *offset start*, calcula la longitud del segmento y procesa los píxeles dentro de ese rango aplicando el mismo algoritmo que en la versión monohilo:

```
1  void* thread_filter(void* args){
2      filter_args_t* tArgs = (filter_args_t*) args;
3
4      data_t *pR = tArgs->pRsrc + tArgs->start;
5      data_t *pG = tArgs->pGsrc + tArgs->start;
6      data_t *pB = tArgs->pBsrc + tArgs->start;
7      data_t *pRdst = tArgs->pRdst + tArgs->start;
8      data_t *pGdst = tArgs->pGdst + tArgs->start;
9      data_t *pBdst = tArgs->pBdst + tArgs->start;
10
11     uint length = tArgs->end - tArgs->start;
12
13     // Procesamiento análogo al filtro monohilo
14
15     pthread_exit(NULL);
16 }
```

La función `execute_multithreaded_filter()` se encarga de coordinar la ejecución paralela. Para ello, calcula los píxeles por hilo, crea los hilos con `pthread_create()` y espera a que terminen con `pthread_join()`. El último hilo se encarga de procesar cualquier píxel restante en caso de que el total no sea divisible por el número de hilos:


```

1 void execute_multithreaded_filter(filter_args_t* filter_args, uint
  pixelsPerThread) {
2   pthread_t threads[NUM_THREADS];
3   filter_args_t thread_args[NUM_THREADS];
4
5   for (uint t = 0; t < NUM_THREADS; t++) {
6     thread_args[t] = *filter_args;
7
8     thread_args[t].start = t * pixelsPerThread;
9     thread_args[t].end = (t == NUM_THREADS - 1) ?
filter_args->pixelCount : (t + 1) * pixelsPerThread;
10
11     if(pthread_create(&threads[t], NULL, thread_filter,
&thread_args[t]) != 0){
12       fprintf(stderr, "Error creating thread");
13       exit(EXIT_FAILURE);
14     }
15   }
16
17   for (uint t = 0; t < NUM_THREADS; t++) {
18     if (pthread_join(threads[t], NULL) != 0) {
19       fprintf(stderr, "Error joining thread %d\n", t);
20       exit(EXIT_FAILURE);
21     }
22   }
23 }

```

Finalmente, en la función principal, se calcula el número de píxeles por hilo y se llama a `execute_multithreaded_filter()` para iniciar el procesamiento paralelo. Esta función se ejecutará el número de veces fijado para medir el tiempo total de ejecución. A continuación, se muestran las imágenes resultantes:



Figura 5: Imagen original



Figura 6: Añadir imagen SIMD

5. Resultados y Análisis

5.1. Medición del rendimiento

Para la obtención de resultados, se midió el tiempo de ejecución de las distintas versiones del filtro (monohilo, SIMD y multihilo) utilizando la función `clock_gettime()` como se describió anteriormente. Para evitar variaciones debidas a la carga del sistema, se realizaron múltiples ejecuciones (10 repeticiones) y se calculó el tiempo promedio. A continuación se muestran los resultados obtenidos:

Versión	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
Monohilo	10.27	10.54	10.64	10.43	10.49	10.30	10.33	10.38	10.35	10.21
Multihilo	6.72	6.58	6.57	6.60	6.54	6.49	6.47	6.63	6.56	6.72
SIMD	13.96	14.22	13.98	14.13	13.70	13.82	14.12	13.71	14.10	14.29

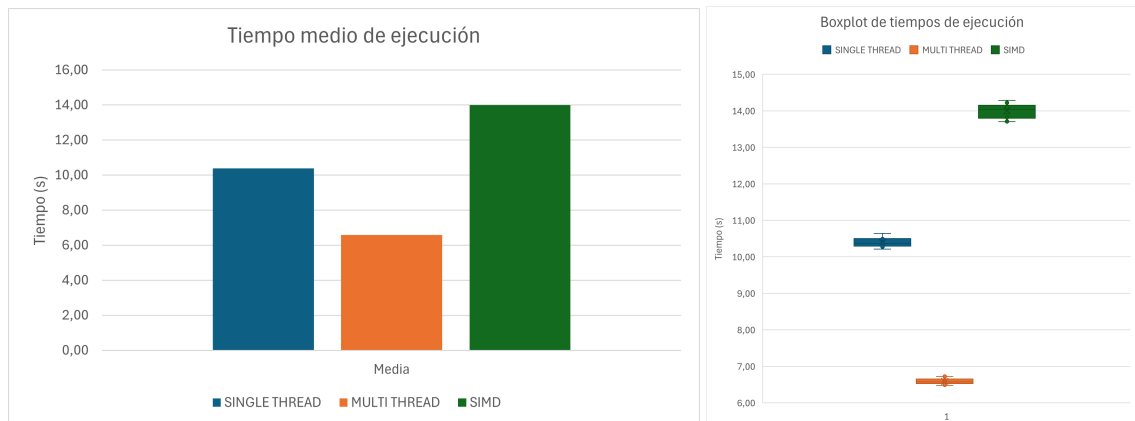
A partir de los datos obtenidos, se calcularon una serie de medidas estadísticas para resumir la información, aplicando medidas de centralización y dispersión. Cabe destacar que, por la presencia de algunos *outliers*, la aproximación del intervalo de confianza mediante el *Teorema Central del Límite* ha dado valores algo imprecisos, por lo que se ha añadido un calculo más preciso empleando la distribución *t-Student* (identificado como *IC - TS*).

Versión	Media (s)	DT (s)	IC - TCL (s)	IC - TS (s)	Product. ($\frac{\text{tareas}}{\text{min}}$)
Monohilo	10.39	0.13	[10.13, 10.66]	[10.35, 10.44]	5.77
Multihilo	6.59	0.08	[6.42, 6.76]	[6.56, 6.62]	9.11
SIMD	14.00	0.21	[13.59, 14.42]	[13.93, 14.08]	4.29

A partir de estos datos, se han calculado las respectivas aceleraciones de las versiones optimizadas respecto a la versión monohilo:

$$\text{Aceleración Multihilo} = \frac{10.39}{6.59} \approx 1.58 \quad \text{Aceleración SIMD} = \frac{10.39}{14.00} \approx 0.74$$

A modo de resumen visual, se presentan los siguientes gráficos que ilustran los tiempos de ejecución y las aceleraciones obtenidas:



5.2. Análisis de la versión multihilo

La implementación multihilo ha mostrado una mejora de rendimiento de aproximadamente 1.58 respecto a la versión monohilo. Aunque esta mejora es significativa, está muy lejos del teórico ideal de x16 aunque se hayan utilizado 8 hilos por motivos que se tratarán más adelante. Esto resulta en una eficiencia del 19.75 %, lo cual indica que existen cuellos de botella en algunos puntos del proceso. Las posibles causas incluyen:

- **Contención de recursos:** Aunque cada hilo trabaja sobre un segmento independiente de la imagen, a partir de cierto número de hilos aparece contención en la memoria caché. Cuando demasiados hilos acceden simultáneamente a regiones de memoria cercanas aumenta el número de fallos de caché. Como el rendimiento pasa a estar limitado por el subsistema de memoria, hemos decidido limitar el número de hilos a 8.
- **Overhead de creación/destrucción de hilos:** En cada iteración se crean y destruyen 8 hilos, lo que introduce un coste adicional en el tiempo total. Se ha mantenido este enfoque por indicaciones presentes en la rúbrica de evaluación.
- **Tamaño de trabajo insuficiente por hilo:** Dado que la imagen no es extremadamente grande, la cantidad de trabajo asignada a cada hilo puede ser insuficiente para amortiguar el overhead de gestión de hilos ya que las operaciones de sincronización pueden suponer un tiempo considerable en comparación con las operaciones de suma/multiplicación realizadas.

5.3. Análisis de la versión SIMD

Los resultados de la versión SIMD son especialmente sorprendentes: en lugar de mejorar el rendimiento, esta implementación resulta **0.74x más lenta** que la versión monohilo original. Se ha comprobado en varios ordenadores con diferentes procesadores y, en todos, los resultados son similares. Se ha verificado que todos ellos tuvieran soporte para las instrucciones necesarias y que el código estuviera optimizado. Algunas posibles causas de este comportamiento incluyen:

- **Overhead en la gestión de memoria:** La utilización de memoria alineada y las operaciones de carga/almacenamiento SIMD pueden introducir un overhead que no compensa la paralelización a nivel de datos, especialmente si el tamaño de los datos no es lo suficientemente grande.
- **Limitaciones del compilador:** Es posible que el compilador no esté optimizando adecuadamente el código SIMD, o que las instrucciones generadas no sean las más eficientes para el hardware específico.

Además, hemos probado a realizar el algoritmo utilizando la operación `_mm_fmadd_pd()` en lugar de `_mm_add_pd()`, pero la imagen generada era distinta respecto a la monohilo y multihilo. Esto creemos que se debe a que la instrucción `_mm_fmadd_pd()` realiza la suma y la multiplicación en una operación combinada, es por ello que solo redondea una vez. A diferencia de hacer una instrucción de suma y otra de multiplicación, que redondea dos veces, lo que provoca pequeñas diferencias en el resultado final.

ANEXOS

A. Flujo de trabajo y metodología de desarrollo

Todo el desarrollo del proyecto se encuentra disponible en el repositorio de *GitHub*: [ver repositorio](#). Para garantizar un trabajo colaborativo eficiente, se estableció un flujo de trabajo común basado en las herramientas *Git* y *GitHub* como elementos centrales de control de versiones. Los aspectos clave son:

- **Repositorio compartido:** Contiene el código, la documentación y los recursos del proyecto.
- **Issues:** Cada tarea se define como una *issue* en *GitHub*, asignada a un responsable con una fecha límite.
- **Project Board:** Se emplea un tablero *Kanban* con las columnas *To Do*, *In Progress*, *Review* y *Done* para visualizar el progreso del trabajo.
- **Milestones:** Cada entrega del proyecto se asocia a un *milestone* que agrupa las *issues* relacionadas.
- **Pull Requests:** Todas las contribuciones se integran mediante *pull requests*, revisadas y aprobadas por, al menos, tres de los cuatro miembros del equipo antes de fusionarse en la rama principal.
- **Documentación:** Se conserva en la carpeta `docs/`.

A.1. Metodología

El equipo adoptó una metodología ágil adaptada al contexto académico, inspirada en un enfoque híbrido entre *Scrum* y *Kanban*. Los elementos principales de esta metodología son:

- La planificación se organiza en **iteraciones** (*milestones*) que corresponden a las entregas del proyecto.
- Las tareas se dividen en **issues** manejables, priorizadas según su relevancia y dependencias, y se asignan a los miembros del equipo.
- El **Project Board** facilita la visualización del progreso y la gestión eficiente de las tareas.
- La revisión colectiva de los *pull requests* promueve la calidad del código y el aprendizaje conjunto.
- Las dependencias entre tareas se gestionan mediante etiquetas y referencias cruzadas en *GitHub*.